

Verilog 基础模块

黑金动力社区 2018-01-18

1 简介

本文主要介绍 verilog 基础模块，夯实基础，对深入学习 FPGA 会有很大帮助。

2 数据类型

2.1 常量

整数：整数可以用二进制 b 或 B，八进制 o 或 O，十进制 d 或 D，十六进制 h 或 H 表示，例如，8'b00001111 表示 8 位位宽的二进制整数，4'ha 表示 4 位位宽的十六进制整数。

x 和 z：x 代表不定值，z 代表高阻值，例如，5'b00x11，第三位不定值，3'b00z 表示最低位为高阻值。

下划线：在位数过长时可以用来分割位数，提高程序可读性，如 8'b0000_1111

参数 parameter：parameter 可以用标识符定义常量，运用时只使用标识符即可，提高可读性及维护性，如定义 parameter width = 8；定义寄存器 reg [width-1:0] a；即定义了 8 位宽度的寄存器。

参数的传递：在一个模块中如果有定义参数，在其他模块调用此模块时可以传递参数，并可以修改参数，如下所示，在 module 后用# () 表示。

例如定义模块如下

```
module rom
#(
    parameter depth = 15,
    parameter width = 8
)
(
    input [depth-1:0] addr ,
    input [width-1:0] data ,
    output result
) ;
```

调用模块

```
module top() ;

    wire [31:0] addr ;
    wire [15:0] data ;
    wire result ;

    rom
    #(
        .depth(32),
        .width(16)
    )
```

```

endmodule

r1
(
  .addr(addr) ,
  .data(data) ,
  .result(result)
) ;
endmodule

```

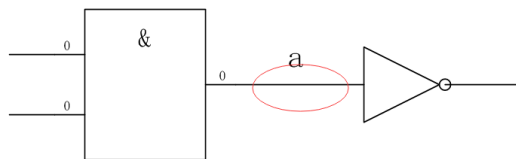
Parameter 可以用于模块间的参数传递，而 localparam 仅用于本模块内使用，不能用于参数传递。Localparam 多用于状态机状态的定义。

2.2 变量

变量是指程序运行时可以改变其值的量，下面主要介绍几个常用了变量类型。

2.2.1 Wire 型

Wire 类型变量，也叫网络类型变量，用于结构实体之间的物理连接，如门与门之间，不能储值，用连续赋值语句 assign 赋值，定义为 wire [n-1:0] a；其中 n 代表位宽，如定义 wire a；assign a = b；是将 b 的结点连接到连线 a 上。如下图所示，两个实体之间的连线即是 wire 类型变量。



2.2.2 Reg 型

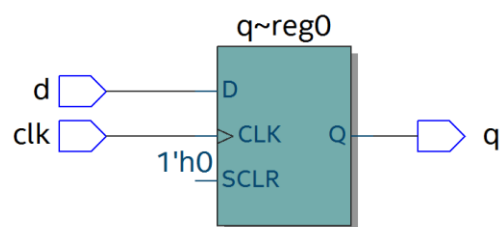
Reg 类型变量，也称为寄存器变量，可用来储存值，必须在 always 语句里使用。其定义为 reg [n-1:0] a；表示 n 位位宽的寄存器，如 reg [7:0] a；表示定义 8 位位宽的寄存器 a。如下所示定义了寄存器 q，生成的电路为时序逻辑，右图为其结构，为 D 触发器。

```

module top(d, clk, q) ;
input d ;
input clk ;
output reg q ;

always @(posedge clk)
begin
  q <= d ;
end
endmodule

```



也可以生成组合逻辑，如数据选择器，敏感信号没有时钟，定义了 reg Mux，最终生成电路为组合逻辑。

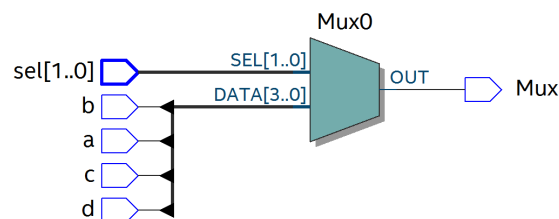
```

module top(a, b, c, d, sel, Mux) ;
input  a ;
input  b ;
input  c ;
input  d ;
input [1:0] sel ;
output reg Mux ;

always @(sel or a or b or c or d)
begin
    case(sel)
        2'b00 : Mux = a ;
        2'b01 : Mux = b ;
        2'b10 : Mux = c ;
        2'b11 : Mux = d ;
    endcase
end

endmodule

```



2.2.3 Memory 型

可以用 memory 类型来定义 RAM,ROM 等存储器，其结构为 reg [n-1:0] 存储器名[m-1:0]，意义为 m 个 n 位宽度的寄存器。例如，reg [7:0] ram [255:0]表示定义了 256 个 8 位寄存器，256 也即是存储器的深度，8 为数据宽度。

3 运算符

运算符可分为以下几类：

- (1) 算术运算符 (+ , - , * , / , %)
- (2) 赋值运算符 (= , <=)
- (3) 关系运算符 (> , < , >= , <= , == , !=)
- (4) 逻辑运算符 (&& , || , !)
- (5) 条件运算符 (? :)
- (6) 位运算符 (~ , | , ^ , & , ^~)

(7) 移位运算符 (<<, >>)

(8) 拼接运算符 ({})

3.1 算术运算符

“+”(加法运算符), “-”(减法运算符), “*” (乘法运算符), “/” (除法运算符, 如 $7/3=2$), “%” (取模运算符, 也即求余数, 如 $7\%3=1$, 余数为 1)

3.2 赋值运算符

“=”阻塞赋值, “<=”非阻塞赋值。阻塞赋值为执行完一条赋值语句, 再执行下一条, 可理解为顺序执行, 而且赋值是立即执行; 非阻塞赋值可理解为并行执行, 不考虑顺序, 在 always 块语句执行完成后, 才进行赋值。如下面的阻塞赋值:

代码如下:

激励文件如下

```
module top(din,a,b,c,clk);
input din;
input clk;
output reg a,b,c;

always @(posedge clk)
begin
    a = din;
    b = a;
    c = b;
end

endmodule
```

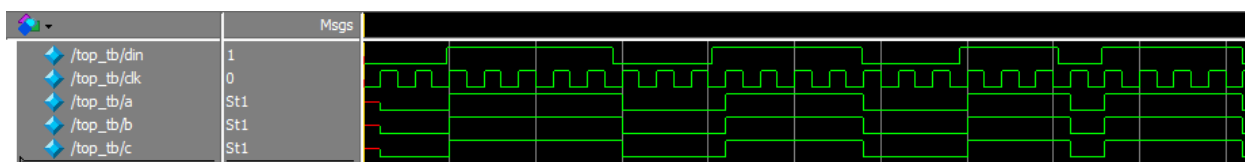
```
`timescale 1 ns/1 ns
module top_tb() ;
reg din ;
reg clk ;
wire a,b,c ;

initial
begin
    din = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        din = ~din ;
    end
end

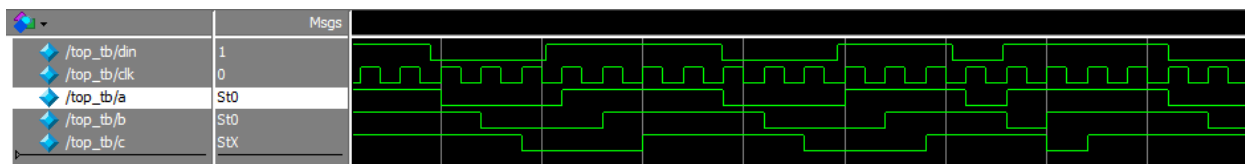
always #10 clk = ~clk ;

top t0(.din(din),.a(a),.b(b),.c(c),.clk(clk)) ;
endmodule
```

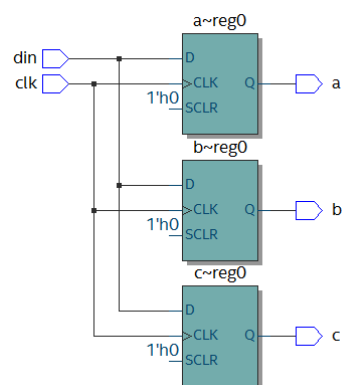
可以从仿真结果看到, 在 clk 的上升沿, a 的值等于 din, 并立即赋给 b, b 的值赋给 c。



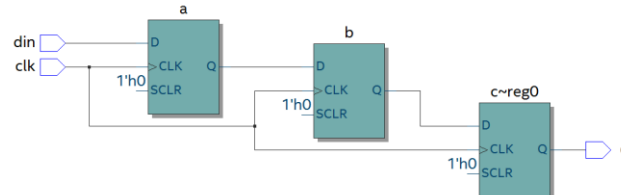
如果改为非阻塞赋值，仿真结果如下，在 clk 上升沿，a 的值没有立即赋值给 b，b 为 a 原来的值，同样，c 为 b 原来的值



可以从两者的 RTL 图看出明显不同：



阻塞赋值 RTL 图



非阻塞赋值 RTL 图

一般情况下，在时序逻辑电路中使用非阻塞赋值，可避免仿真时出现竞争冒险现象；在组合逻辑中使用阻塞赋值，执行赋值语句后立即改变；在 assign 语句中必须用阻塞赋值。

3.3 关系运算符

用于表示两个操作数之间的关系，如 $a > b$ ， $a < b$ ，多用于判断条件，例如：

```
If (a>=b) q <= 1'b1;
```

```
else q <= 1'b0;
```

表示如果 a 的值大于等于 b 的值，则 q 的值为 1，否则 q 的值为 0

3.4 逻辑运算符

“&&”（两个操作数逻辑与），“||”（两个操作数逻辑或），“!”（单个操作数逻辑非）例如：

If (a>b && c<d) 表示条件为 $a > b$ 并且 $c < d$; if (!a) 表示条件为 a 的值不为 1，也就是 0。

3.5 条件运算符

“?:”为条件判断，类似于 if else，例如 `assign a = (i>8)?1'b1:1'b0` ;判断 i 的值是否大于 8，如果大于 8 则 a 的值为 1，否则为 0。

3.6 位运算符

“~”按位取反，“|”按位或，“^”按位异或，“&”按位与，“^~”按位同或，除了“~”只需要一个操作数外，其他几个都需要两个操作数，如 `a&b`，`a|b`。具体应用在后面的组合逻辑一节中有讲解。

3.7 移位运算符

“<<”左移位运算符，“>>”右移位运算符，如 `a<<1` 表示，向左移 1 位，`a>>2`，向右移两位。

3.8 拼接运算符

“{ }”拼接运算符，将多个信号按位拼接，如 `{a[3:0], b[1:0]}`，将 a 的低 4 位，b 的低 2 位拼接成 6 位数据。另外，`{n{a[3:0]}}` 表示将 n 个 a[3:0] 拼接，`{n{1'b0}}` 表示 n 位的 0 拼接。如 `{8{1'b0}}` 表示为 `8'b0000_0000`。

3.9 优先级

各种运算符的优先级如下：

! ~	高优先级
/ * %	
+ -	
<< >>	
< <= > >=	
== !=	
&	
^ ^~	
&&	
?:	
	低优先级

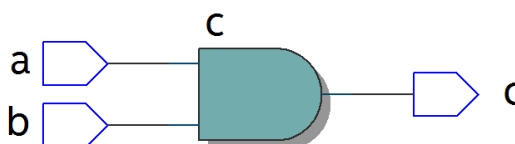
4 组合逻辑

本节主要介绍组合逻辑，组合逻辑电路的特点是任意时刻的输出仅仅取决于输入信号，输入信号变化，输出立即变化，不依赖于时钟。

4.1 与门

在 verilog 中以 “&” 表示按位与，如 $c=a\&b$ ，真值表如下，在 a 和 b 都等于 1 时结果才为 1，RTL 表示如右图

输入		输出
a	b	c
0	0	0
0	1	0
1	0	0
1	1	1



代码实现如下：

```
module top(a, b, c) ;
input  a ;
input  b ;
output c ;

assign c = a & b ;
endmodule
```

激励文件如下：

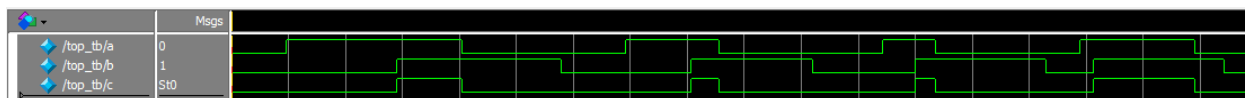
```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire c ;

initial
begin
a = 0 ;
b = 0 ;
forever
begin
#({$random}%100)
a = ~a ;
#({$random}%100)
b = ~b ;
end
end

top  t0(.a(a), .b(b), .c(c)) ;

endmodule
```

仿真结果如下：

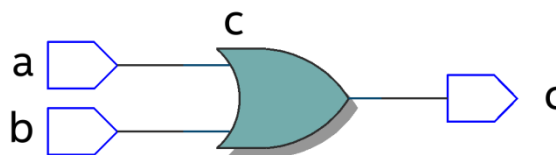


如果 a 和 b 的位宽大于 1，例如定义 input [3:0] a, input [3:0] b，那么 a&b 则指 a 与 b 的对应位相与。如 a[0]&b[0], a[1]&b[1]。

4.2 或门

在 verilog 中以 “|” 表示按位或，如 c = a|b，真值表如下，在 a 和 b 都为 0 时结果才为 0。

输入		输出
a	b	c
0	0	0
0	1	1
1	0	1
1	1	1



代码实现如下：

```
module top(a, b, c) ;
input  a ;
input  b ;
output c ;

assign c = a | b ;
endmodule
```

激励文件如下

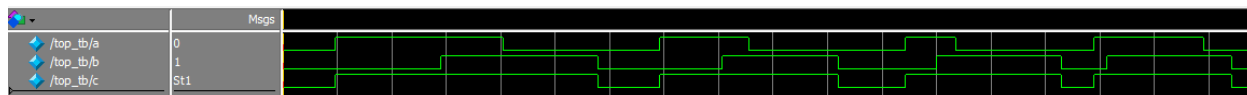
```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire c ;

initial
begin
    a = 0 ;
    b = 0 ;
    forever
    begin
        #({$random}%100)
        a = ~a ;
        #({$random}%100)
        b = ~b ;
    end
end

top  t0(.a(a), .b(b), .c(c)) ;

endmodule
```

仿真结果如下：

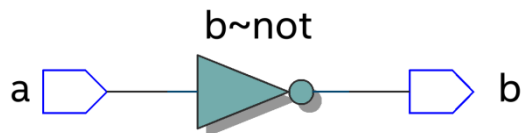


同理，位宽大于 1，则是按位或。

4.3 非门

在 verilog 中以 “~” 表示按位取反，如 $b = \sim a$ ，真值表如下，b 等于 a 的相反数。

输入	输出
a	b
0	1
1	0



代码实现如下：

```
module top(a, b) ;
input  a ;
output b ;

assign b = ~a ;
endmodule
```

激励文件如下：

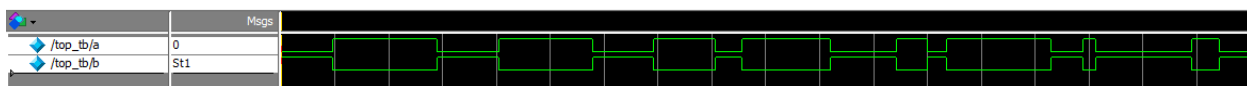
```
`timescale 1 ns/1 ns
module top_tb() ;
reg  a ;
wire b ;

initial
begin
    a = 0 ;
    forever
    begin
        #({$random}%100)
        a = ~a ;
    end
end

top  t0(.a(a), .b(b)) ;

endmodule
```

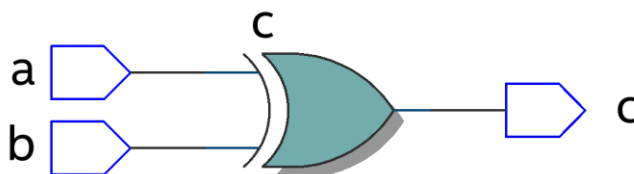
仿真结果如如下：



4.4 异或

在 verilog 中以 “^” 表示异或，如 $c = a \wedge b$ ，真值表如下，当 a 和 b 相同时，输出为 0。

输入		输出
a	b	c
0	0	0
0	1	1
1	0	1
1	1	0



代码实现如下：

```
module top(a, b, c) ;
input  a ;
input  b ;
output c ;

assign c = a ^ b ;
endmodule
```

激励文件如下：

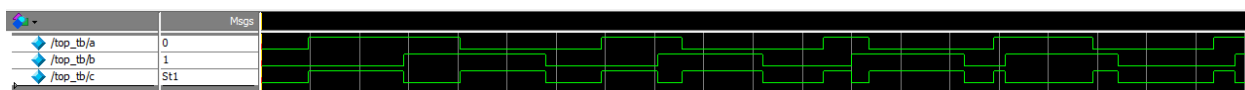
```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire c ;

initial
begin
a = 0 ;
b = 0 ;
forever
begin
#({$random}%100)
a = ~a ;
#({$random}%100)
b = ~b ;
end
end

top  t0(.a(a), .b(b), .c(c)) ;

endmodule
```

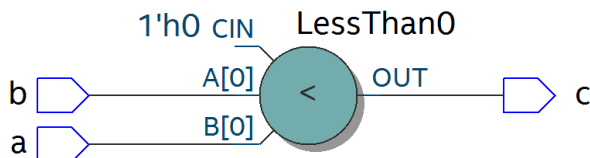
仿真结果如下：



4.5 比较器

在 verilog 中以大于 ">", 等于"==" , 小于"<" , 大于等于">=" , 小于等于"<=" , 不等于"!="表示, 以大于举例, 如 $c = a > b$; 表示如果 a 大于 b, 那么 c 的值就为 1, 否则为 0。真值表如下：

输入		输出
a	b	c
0	0	0
0	1	0
1	0	1
1	1	0



代码实现如下：

```
module top(a, b, c) ;
input  a ;
input  b ;
output c ;

assign c = a > b ;
endmodule
```

激励文件如下：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire c ;

initial
begin
```

```

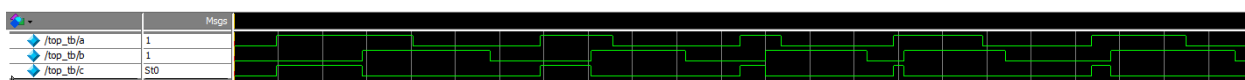
a = 0 ;
b = 0 ;
forever
begin
    #({$random}%100)
    a = ~a ;
    #({$random}%100)
    b = ~b ;
end
end

top  t0(.a(a), .b(b), .c(c)) ;

endmodule

```

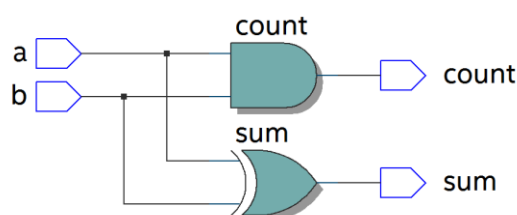
仿真结果如下：



4.6 半加器

半加器和全加器是算术运算电路中的基本单元，由于半加器不考虑从低位来的进位，所以称之为半加器，sum 表示相加结果，count 表示进位，真值表可表示如下：

输入		输出	
a	b	sum	count
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



可根据真值表写出代码如下：

```

module top(a, b, sum, count) ;
input  a ;
input  b ;
output sum ;
output count ;

assign sum = a ^ b ;
assign count = a & b ;

endmodule

```

激励文件如下：

```

`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire sum ;
wire count ;

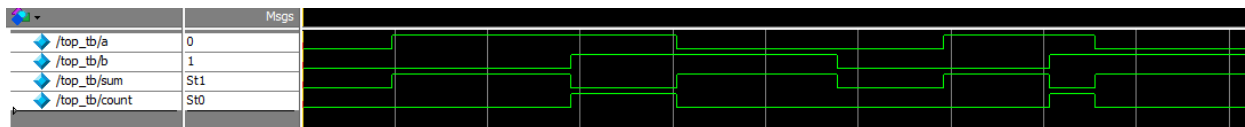
initial
begin
    a = 0 ;
    b = 0 ;
    forever
    begin
        #({$random}%100)
        a = ~a ;
        #({$random}%100)
        b = ~b ;
    end
end
end

```

```
top t0(.a(a), .b(b),
      .sum(sum), .count(count)) ;
```

```
endmodule
```

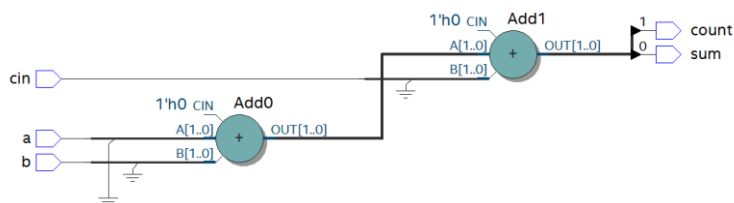
仿真结果如下：



4.7 全加器

而全加器需要加上低位来的进位信号 cin，真值表如下：

输入			输出	
cin	a	b	sum	count
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



代码如下：

```
module top(cin, a, b, sum, count) ;
input cin ;
input a ;
input b ;
output sum ;
output count ;

assign {count,sum} = a + b + cin ;

endmodule
```

激励文件如下：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
reg cin ;
wire sum ;
wire count ;

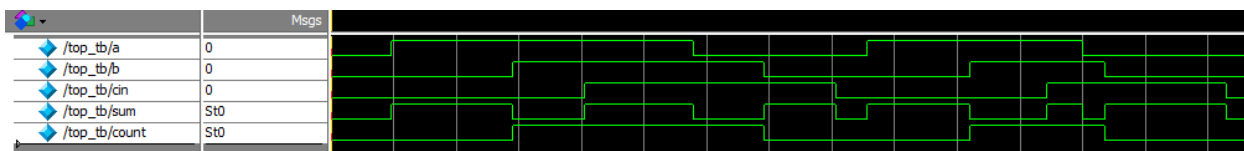
initial
begin
a = 0 ;
b = 0 ;
cin = 0 ;
forever
begin
#({$random}%100)
a = ~a ;
#({$random}%100)
b = ~b ;
#({$random}%100)
cin = ~cin ;

end
end

top t0(.cin(cin), .a(a), .b(b),
      .sum(sum), .count(count)) ;

endmodule
```

仿真结果如下：



4.8 乘法器

乘法的表示也很简单，利用“*”即可，如 $a*b$ ，举例代码如下：

```

module top(a, b, c) ;
input  [1:0] a ;
input  [1:0] b ;
output [3:0] c ;

assign c = a * b ;
endmodule

`timescale 1 ns/1 ns
module top_tb() ;
reg [1:0] a ;
reg [1:0] b ;
wire [3:0] c ;

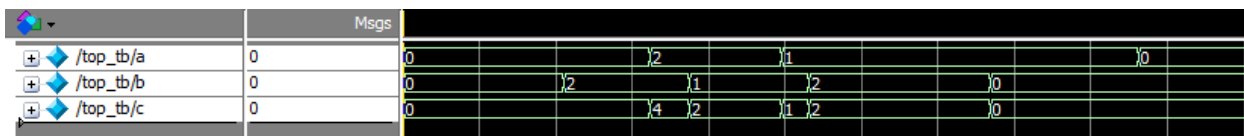
initial
begin
    a = 0 ;
    b = 0 ;
    forever
    begin
        #({$random}%100)
        a = ~a ;
        #({$random}%100)
        b = ~b ;
    end
end

top  t0(.a(a), .b(b), .c(c)) ;

endmodule

```

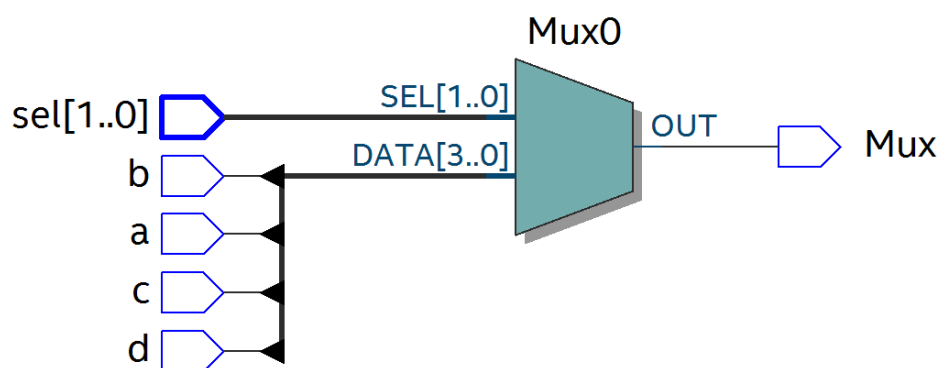
仿真结果如下：



4.9 数据选择器

在 verilog 中经常会用到数据选择器，通过选择信号，选择不同的输入信号输出到输出端，如下图所示真值表，四选一数据选择器，sel[1:0]为选择信号，a,b,c,d 为输入信号，Mux 为输出信号。

选择信号		输入				输出
sel[0]	sel[1]	a	b	c	d	Mux
0	0	x	x	x	x	a
0	1	x	x	x	x	b
1	0	x	x	x	x	c
1	1	x	x	x	x	d



代码如下：

```
module top(a, b, c, d, sel, Mux) ;
input  a ;
input  b ;
input  c ;
input  d ;

input [1:0] sel ;

output reg Mux ;

always @(sel or a or b or c or d)
begin
    case(sel)
        2'b00 : Mux = a ;
        2'b01 : Mux = b ;
        2'b10 : Mux = c ;
        2'b11 : Mux = d ;
    endcase
end

endmodule
```

激励文件如下：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
reg c ;
reg d ;
reg [1:0] sel ;
wire Mux ;

initial
begin
    a = 0 ;
    b = 0 ;
    c = 0 ;
    d = 0 ;
    forever
    begin
        #({$random}%100)
        a = {$random}%3 ;
        #({$random}%100)
        b = {$random}%3 ;
        #({$random}%100)
        c = {$random}%3 ;
        #({$random}%100)
        d = {$random}%3 ;
    end
end
```

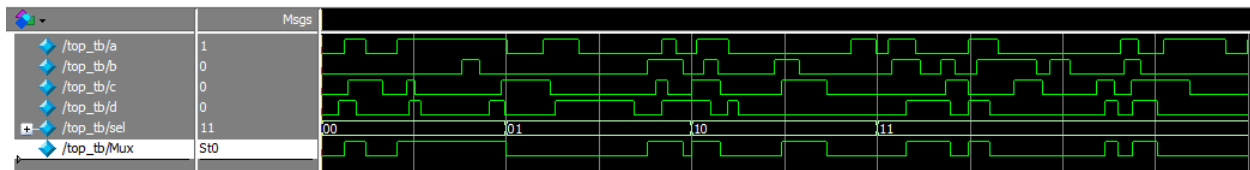
```
end

initial
begin
    sel = 2'b00 ;
    #2000 sel = 2'b01 ;
    #2000 sel = 2'b10 ;
    #2000 sel = 2'b11 ;
end

top
t0(.a(a), .b(b), .c(c), .d(d), .sel(sel),
.Mux(Mux)) ;

endmodule
```

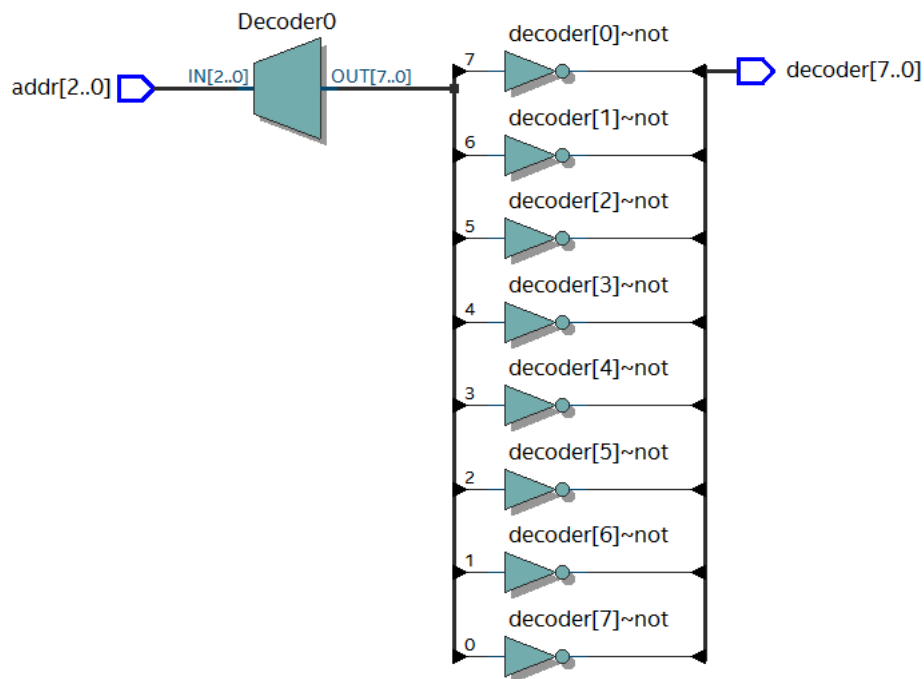
仿真结果如下



4.10 3-8 译码器

3-8 译码器是一个很常用的器件，其真值表如下所示，根据 A2,A1,A0 的值，得出不同的结果。

输入			输出							
A2	A1	A0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0



代码如下：

```

module top(addr, decoder) ;
input  [2:0] addr ;
output reg [7:0] decoder ;

always @(addr)
begin
    case (addr)
        3'b000 : decoder = 8'b1111_1110 ;
        3'b001 : decoder = 8'b1111_1101 ;
        3'b010 : decoder = 8'b1111_1011 ;
        3'b011 : decoder = 8'b1111_0111 ;
        3'b100 : decoder = 8'b1110_1111 ;
        3'b101 : decoder = 8'b1101_1111 ;
        3'b110 : decoder = 8'b1011_1111 ;
        3'b111 : decoder = 8'b0111_1111 ;
    endcase
end

endmodule

```

激励文件如下：

```

`timescale 1 ns/1 ns
module top_tb() ;
reg  [2:0] addr ;
wire  [7:0] decoder ;

initial
begin
    addr = 3'b000 ;
    #2000 addr = 3'b001 ;
    #2000 addr = 3'b010 ;
    #2000 addr = 3'b011 ;
    #2000 addr = 3'b100 ;
    #2000 addr = 3'b101 ;
    #2000 addr = 3'b110 ;
    #2000 addr = 3'b111 ;
end

top
t0(.addr(addr),.decoder(decoder)) ;

endmodule

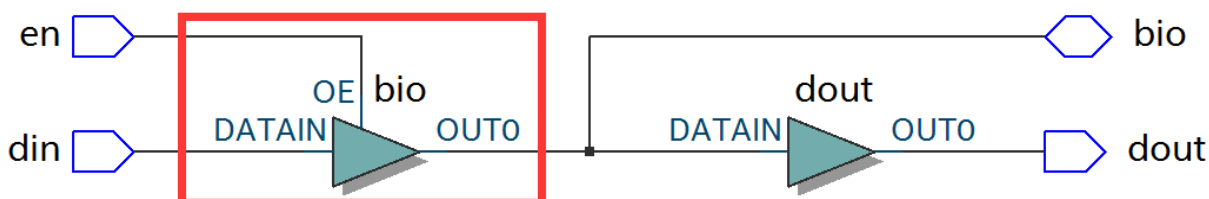
```

仿真结果如下：

	Msgs								
/top_tb/addr	111	000	001	010	011	100	101	110	111
/top_tb/decoder	01111111	11111110	11111101	11111011	11110111	11101111	11011111	10111111	01111111

4.11 三态门

在 FPGA 使用中，经常会用到双向 IO，需要用到三态门，如 $bio = en ? din : 1'bz$ ；其中 en 为使能信号，用于打开关闭三态门，下面的 RTL 图即是实现了双向 IO，可参考代码。激励文件实现两个双向 IO 的对接。



```
module top(en, din, dout, bio) ;
input  din ;
input  en  ;
output dout ;
inout  bio ;

assign bio = en? din : 1'bz ;
assign dout = bio ;

endmodule
```

```
`timescale 1 ns/1 ns
module top_tb() ;
reg en0 ;
reg din0 ;
wire dout0 ;
reg en1 ;
reg din1 ;
wire dout1 ;
wire bio ;

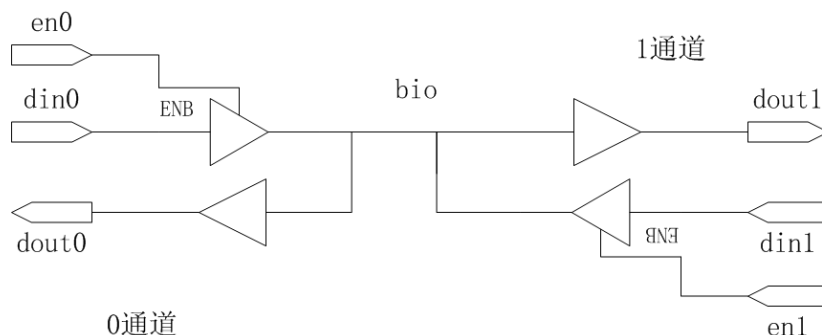
initial
begin
    din0 = 0 ;
    din1 = 0 ;
    forever
    begin
        #({$random}%100)
        din0 = ~din0 ;
        #({$random}%100)
        din1 = ~din1 ;
    end
end

initial
begin
    en0 = 0 ;
    en1 = 1 ;
    #100000
    en0 = 1 ;
    en1 = 0 ;
end

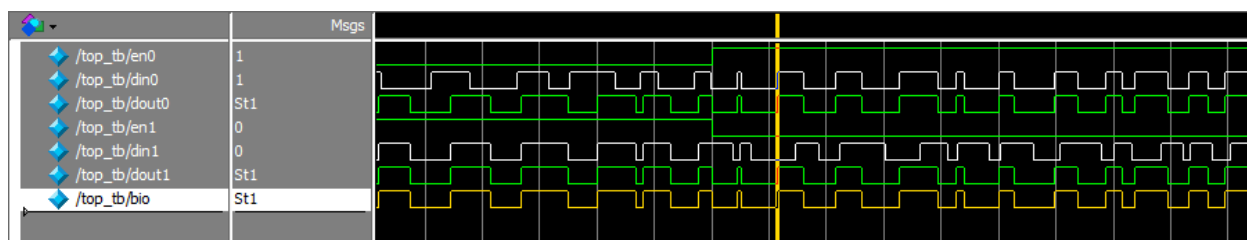
top
t0(.en(en0),.din(din0),.dout(dout0),.bio(bio)) ;
top
t1(.en(en1),.din(din1),.dout(dout1),.bio(bio)) ;

endmodule
```

激励文件结构如下图



仿真结果如下，en0 为 0，en1 为 1 时，1 通道打开，双向 IO bio 就等于 1 通道的 din1，1 通道向外发送数据，0 通道接收数据，dout0 等于 bio；当 en0 为 1，en1 为 0 时，0 通道打开，双向 IO bio 就等于 0 通道的 din0，0 通道向外发送数据，1 通道接收数据，dout1 等于 bio



5 时序逻辑

组合逻辑电路在逻辑功能上特点是任意时刻的输出仅仅取决于当前时刻的输入，与电路原来的状态无关。而时序逻辑在逻辑功能上的特点是任意时刻的输出不仅仅取决于当前的输入信号，而且还取决于电路原来的状态。下面以典型的时序逻辑分析。

5.1 D 触发器

D 触发器在时钟的上升沿或下降沿存储数据，输出与时钟跳变之前输入信号的状态相同。

代码如下

```
module top(d, clk, q) ;
input d ;
input clk ;
output reg q ;
always @(posedge clk)
```

激励文件如下

```
`timescale 1 ns/1 ns
module top_tb() ;
reg d ;
reg clk ;
wire q ;
```

```

begin
    q <= d ;
end

endmodule

initial
begin
    d = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

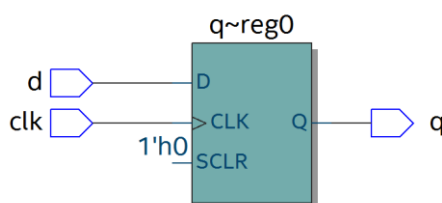
always #10 clk = ~clk ;

top t0(.d(d),.clk(clk),.q(q)) ;

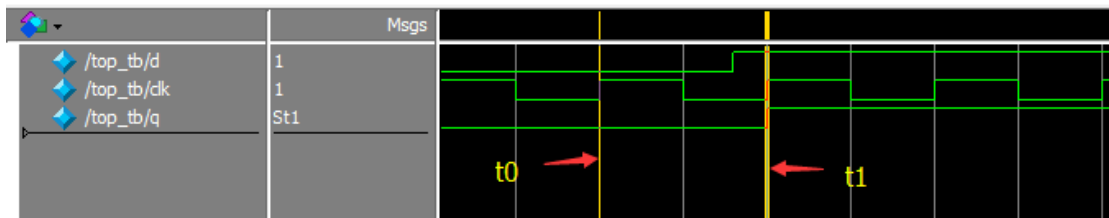
endmodule

```

RTL 图表示如下

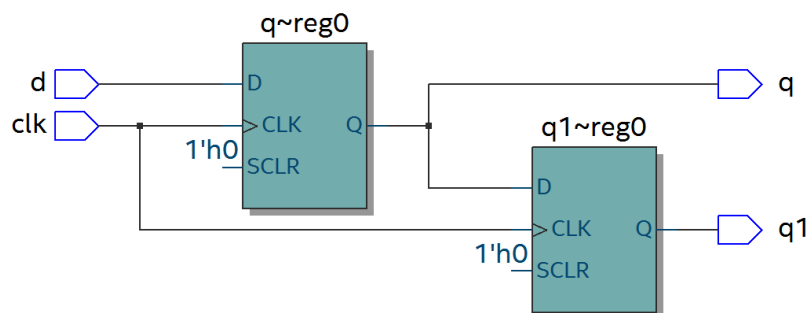


仿真结果如下，可以看到在 t_0 时刻时， d 的值为 0，则 q 的值也为 0；在 t_1 时刻 d 发生了变化，值为 1，那么 q 相应也发生了变化，值变为 1。可以看到在 t_0 - t_1 之间的一个时钟周期内，无论输入信号 d 的值如何变化， q 的值是保持不变的，也就是有存储的功能，保存的值为在时钟的跳变沿时 d 的值。



5.2 两级 D 触发器

软件是按照两级 D 触发器的模型进行时序分析的，具体可以分析在同一时刻两个 D 触发器输出的数据有何不同，其 RTL 图如下：



代码如下：

```
module top(d, clk, q, q1) ;
input  d ;
input  clk ;
output reg q ;
output reg q1 ;

always @(posedge clk)
begin
    q <= d ;
end

always @(posedge clk)
begin
    q1 <= q ;
end

endmodule
```

激励文件如下：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg d ;
reg clk ;
wire q ;
wire q1 ;

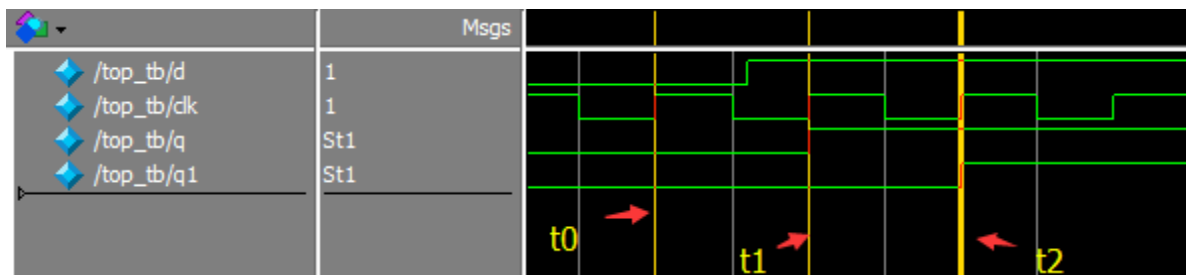
initial
begin
    d = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

always #10 clk = ~clk ;

top
t0(.d(d),.clk(clk),.q(q),.q1(q1)) ;

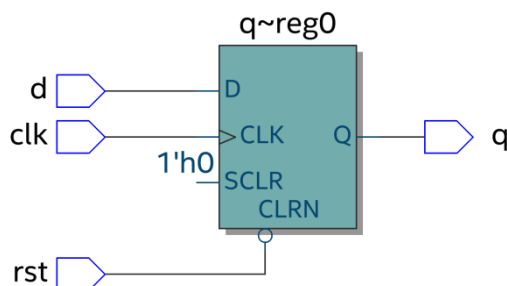
endmodule
```

仿真结果如下，可以看到 t0 时刻，d 为 0，q 输出为 0，t1 时刻，q 随着 d 的数据变化而变化，而此时时钟跳变之前 q 的值仍为 0，那么 q1 的值仍为 0，t2 时刻，时钟跳变前 q 的值为 1，则 q1 的值相应为 1，q1 相对于 q 落后一个周期。



5.3 带异步复位的 D 触发器

异步复位是指独立于时钟，一旦异步复位信号有效，就触发复位操作。这个功能在写代码时会经常用到，用于给信号复位，初始化。其 RTL 图如下：



代码如下，注意要把异步复位信号放在敏感列表里，如果是低电平复位，即为 negedge，如果是高电平复位，则是 posedge

```
module top(d, rst, clk, q) ;
input d ;
input rst ;
input clk ;
output reg q ;

always @(posedge clk or negedge rst)
begin
    if (rst == 1'b0)
        q <= 0 ;
    else
        q <= d ;
    end
endmodule
```

```
`timescale 1 ns/1 ns
module top_tb() ;
reg d ;

reg rst ;
reg clk ;
wire q ;

initial
begin
    d = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

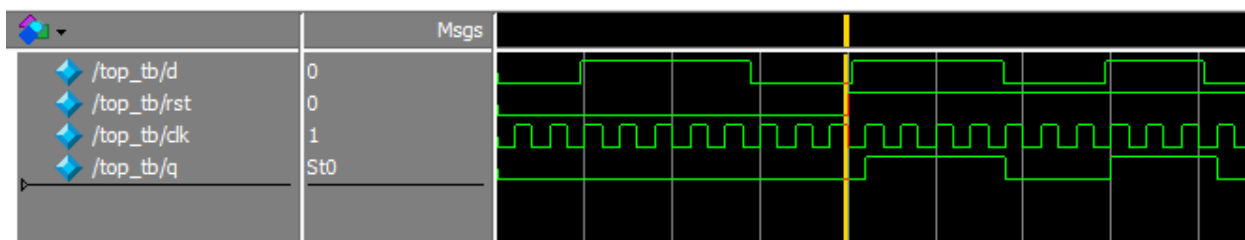
initial
begin
    rst = 0 ;
    #200 rst = 1 ;
end

always #10 clk = ~clk ;

top
t0(.d(d),.rst(rst),.clk(clk),.q(q)) ;

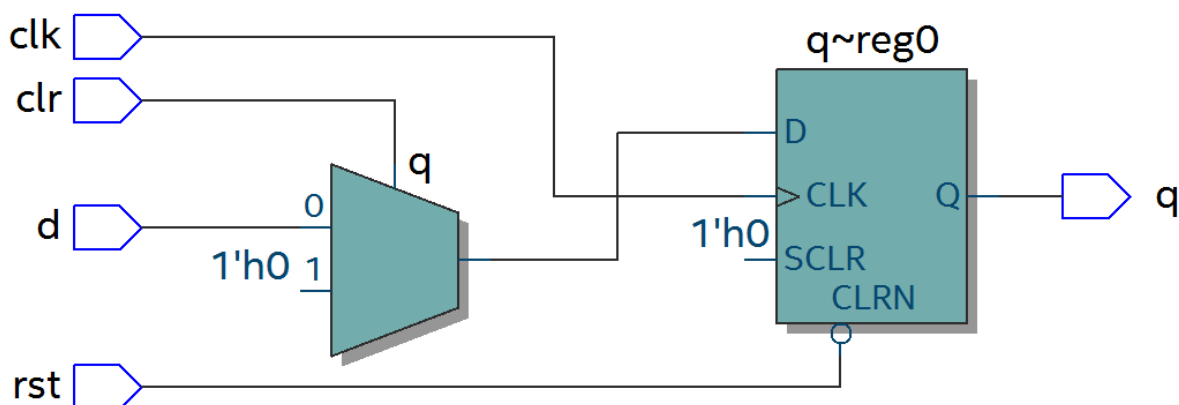
endmodule
```

仿真结果如下，可以看到在复位信号之前，虽然输入信号 d 数据有变化，但由于正处于复位状态，输入信号 q 始终为 0，在复位之后 q 的值就正常了。



5.4 带异步复位同步清零的 D 触发器

前面讲到异步复位独立于时钟操作，而同步清零则是同步于时钟信号下操作的，当然也不仅限于同步清零，也可以是其他的同步操作，其 RTL 图如下：



代码如下，不同于异步复位，同步操作不能把信号放到敏感列表里

```

module top(d, rst, clr, clk, q) ;
input d ;
input rst ;
input clr ;
input clk ;
output reg q ;

always @(posedge clk or negedge rst)
begin
    if (rst == 1'b0)
        q <= 0 ;
    else if (clr == 1'b1)
        q <= 0 ;
    else
        q <= d ;
end

endmodule

`timescale 1 ns/1 ns
module top_tb() ;
reg d ;
reg rst ;
reg clr ;
reg clk ;
wire q ;

initial
begin
    d = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end
end

```

```

initial
begin
    rst = 0 ;
    clr = 0 ;
    #200 rst = 1 ;
    #200 clr = 1 ;
    #100 clr = 0 ;
end

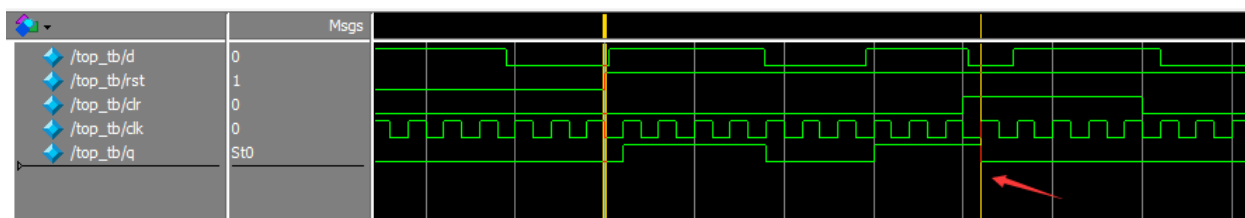
always #10 clk = ~clk ;

top
t0(.d(d),.rst(rst),.clr(clr),.clk(clk),
.q(q)) ;

endmodule

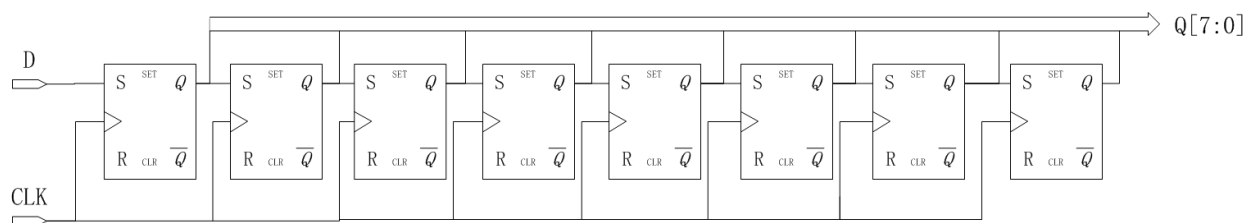
```

仿真结果如下，可以看到 clr 信号拉高后，q 没有立即清零，而是在下个 clk 上升沿之后执行清零操作，也就是 clr 同步于 clk。



5.5 移位寄存器

移位寄存器是指在每个时钟脉冲来时，向左或向右移动一位，由于 D 触发器的特性，数据输出同步于时钟边沿，其结构如下，每个时钟来临，每个 D 触发器的输出 q 等于前一个 D 触发器输出的值，从而实现移位的功能。



代码实现：

```

module top(d, rst, clk, q) ;
input d ;
input rst ;
input clk ;
output reg [7:0] q ;

always @(posedge clk or negedge rst)

```

激励文件：

```

`timescale 1 ns/1 ns
module top_tb() ;
reg d ;

reg rst ;
reg clk ;
wire [7:0] q ;

```

```

begin
    if (rst == 1'b0)
        q <= 0 ;
    else
        q <= {q[6:0], d} ; //向左移位
        //q <= {d, q[7:1]} ; //向右移位
    end
endmodule

initial
begin
    d = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

initial
begin
    rst = 0 ;
    #200 rst = 1 ;
end

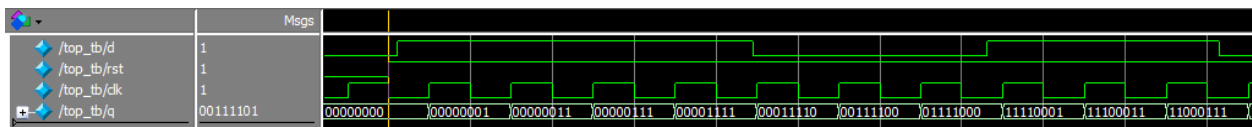
always #10 clk = ~clk ;

top
t0(.d(d),.rst(rst),.clk(clk),.q(q)) ;

endmodule

```

仿真结果如下，可以看到复位之后，每个 clk 上升沿左移一位



5.6 单口 RAM

单口 RAM 的写地址与读地址共用一个地址，代码如下，其中 reg [7:0] ram [63:0]意思是定义了 64 个 8 位宽度的数据。其中定义了 addr_reg，可以保持住读地址，延迟一周期之后将数据送出。

```

module top
(
    input [7:0] data,
    input [5:0] addr,
    input wr,
    input clk,
    output [7:0] q
);

reg [7:0] ram[63:0]; //declare ram
reg [5:0] addr_reg; //addr register

always @ (posedge clk)
begin
    if (wr) //write
        ram[addr] <= data;

    initial
    begin
        data = 0 ;
        addr = 0 ;
        wr = 1 ;
        clk = 0 ;
    end

    always #10 clk = ~clk ;
endmodule

```



```

    addr_reg <= addr;
end

assign q = ram[addr_reg]; //read data
endmodule

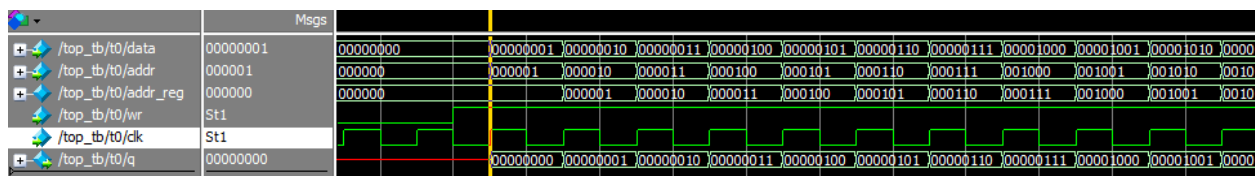
always @(posedge clk)
begin
    data <= data + 1'b1 ;
    addr <= addr + 1'b1 ;
end

top t0(.data(data),
        .addr(addr),
        .clk(clk),
        .wr(wr),
        .q(q)) ;

endmodule

```

仿真结果如下，可以看到 q 的输出与写入的数据一致



5.7 伪双口 RAM

伪双口 RAM 的读写地址是独立的，可以随机选择写或读地址，同时进行读写操作。代码如下，在激励文件中定义了 en 信号，在其有效时发送读地址。

```

module top
(
    input [7:0] data,
    input [5:0] write_addr,
    input [5:0] read_addr,
    input wr,
    input rd,
    input clk,
    output reg [7:0] q
);

reg [7:0] ram[63:0]; //declare ram
reg [5:0] addr_reg; //addr register

always @ (posedge clk)
begin
    if (wr) //write
        ram[write_addr] <= data;
    if (rd) //read
        q <= ram[read_addr];
end

endmodule

`timescale 1 ns/1 ns
module top_tb() ;
reg [7:0] data ;
reg [5:0] write_addr ;
reg [5:0] read_addr ;
reg wr ;
reg rd ;
reg clk ;
reg q ;
wire [7:0] q ;

initial
begin
    data = 0 ;
    write_addr = 0 ;
    read_addr = 0 ;
    wr = 0 ;
    rd = 0 ;
    clk = 0 ;
    #100 wr = 1 ;
    #20 rd = 1 ;
end

always #10 clk = ~clk ;

always @(posedge clk)
begin
    if (wr)
    begin
        data <= data + 1'b1 ;
    end
end

```

```

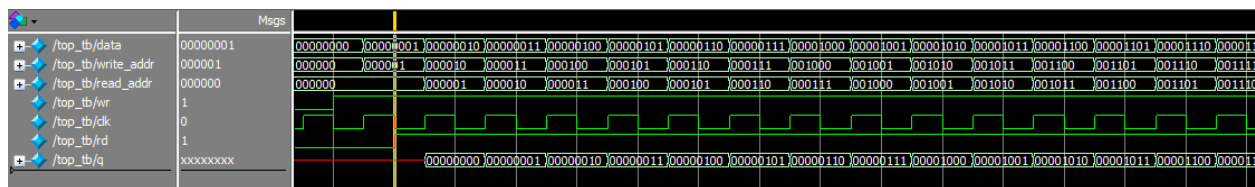
        write_addr <= write_addr + 1'b1 ;
    if (rd)
        read_addr <= read_addr + 1'b1 ;
    end
end

top t0(.data(data),
        .write_addr(write_addr),
        .read_addr(read_addr),
        .clk(clk),
        .wr(wr),
        .rd(rd),
        .q(q)) ;

endmodule

```

仿真结果如下，可以看到在 rd 有效时，对读地址进行操作，读出数据



5.8 真双口 RAM

真双口 RAM 有两套控制线，数据线，允许两个系统对其进行读写操作，代码如下：

```

module top
(
    input [7:0] data_a, data_b,
    input [5:0] addr_a, addr_b,
    input wr_a, wr_b,
    input rd_a, rd_b,
    input clk,
    output reg [7:0] q_a, q_b
);

reg [7:0] ram[63:0]; //declare ram

//Port A
always @ (posedge clk)
begin
    if (wr_a) //write
    begin
        ram[addr_a] <= data_a;
        q_a <= data_a ;
    end
    if (rd_a)
    //read
    q_a <= ram[addr_a];
end

//Port B
always @ (posedge clk)
begin
    `timescale 1 ns/1 ns
    module top_tb() ;
    reg [7:0] data_a, data_b ;
    reg [5:0] addr_a, addr_b ;
    reg wr_a, wr_b ;
    reg rd_a, rd_b ;
    reg clk ;
    wire [7:0] q_a, q_b ;

    initial
    begin
        data_a = 0 ;
        data_b = 0 ;
        addr_a = 0 ;
        addr_b = 0 ;
        wr_a = 0 ;
        wr_b = 0 ;
        rd_a = 0 ;
        rd_b = 0 ;
        clk = 0 ;
        #100 wr_a = 1 ;
        #100 rd_b = 1 ;
    end

    always #10 clk = ~clk ;

    always @(posedge clk)
    begin
        if (wr_a)

```

```

    if (wr_b)                                //write
    begin
        ram[addr_b] <= data_b;
        q_b <= data_b ;
    end
    if (rd_b)
    //read
        q_b <= ram[addr_b];
    end
endmodule

begin
    data_a <= data_a + 1'b1 ;
    addr_a <= addr_a + 1'b1 ;
end
else
begin
    data_a <= 0 ;
    addr_a <= 0 ;
end
end

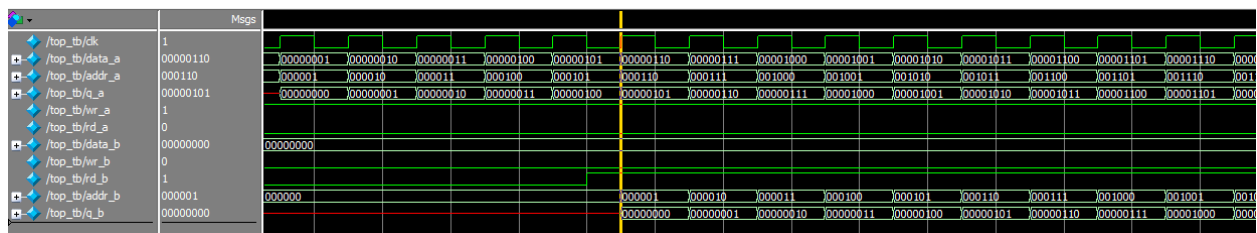
always @(posedge clk)
begin
    if (rd_b)
    begin
        addr_b <= addr_b + 1'b1 ;
    end
    else addr_b <= 0 ;

end

top
t0(.data_a(data_a), .data_b(data_b),
    .addr_a(addr_a), .addr_b(addr_b)
),
    .wr_a(wr_a), .wr_b(wr_b),
    .rd_a(rd_a), .rd_b(rd_b),
    .clk(clk),
    .q_a(q_a), .q_b(q_b)) ;
endmodule

```

仿真结果如下



5.9 单口 ROM

ROM 是用来存储数据的，可以按照下列代码形式初始化 ROM，但这种方法处理大容量的 ROM 就比较麻烦，建议用 FPGA 自带的 ROM IP 核实现，并添加初始化文件。

代码实现

```
module top
(
    input [3:0] addr,
    input clk,
    output reg [7:0] q
);

reg [7:0] rom [15:0] ; //declare rom

always @(addr)
begin
    case(addr)
        4'd0 : rom[addr] = 8'd15 ;
        4'd1 : rom[addr] = 8'd24 ;
        4'd2 : rom[addr] = 8'd100 ;
        4'd3 : rom[addr] = 8'd78 ;
        4'd4 : rom[addr] = 8'd98 ;
        4'd5 : rom[addr] = 8'd105 ;
        4'd6 : rom[addr] = 8'd86 ;
        4'd7 : rom[addr] = 8'd254 ;
        4'd8 : rom[addr] = 8'd76 ;
        4'd9 : rom[addr] = 8'd35 ;
        4'd10 : rom[addr] = 8'd120 ;
        4'd11 : rom[addr] = 8'd85 ;
        4'd12 : rom[addr] = 8'd37 ;
        4'd13 : rom[addr] = 8'd19 ;
        4'd14 : rom[addr] = 8'd22 ;
        4'd15 : rom[addr] = 8'd67 ;
    endcase
end

always @(posedge clk)
begin
    q <= rom[addr] ;
end

endmodule
```

仿真结果如下

激励文件

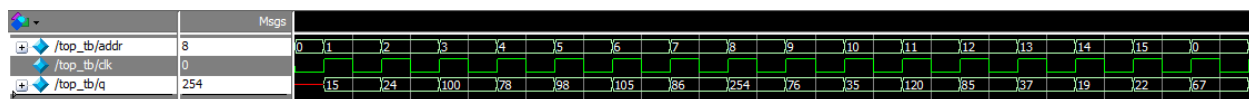
```
`timescale 1 ns/1 ns
module top_tb() ;
    reg [3:0] addr ;
    reg clk ;
    wire [7:0] q ;

    initial
    begin
        addr = 0 ;
        clk = 0 ;
    end

    always #10 clk = ~clk ;

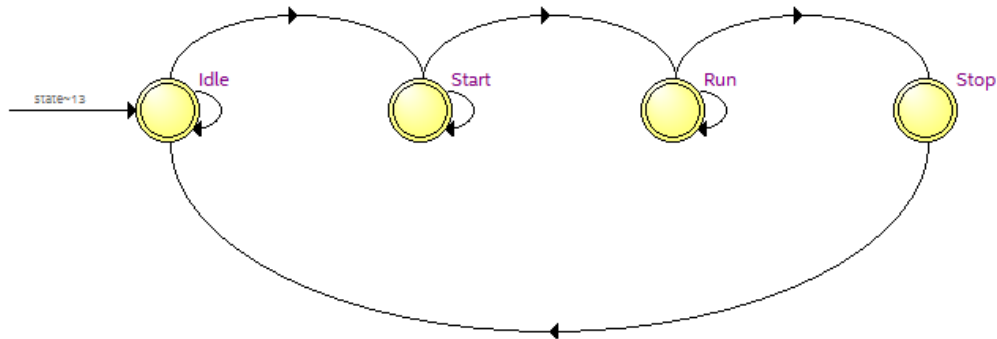
    always @(posedge clk)
    begin
        addr <= addr + 1'b1 ;
    end

    top t0(.addr(addr),
           .clk(clk),
           .q(q)) ;
endmodule
```



5.10 有限状态机

在 verilog 里经常会用到有限状态机，处理相对复杂的逻辑，设定好不同的状态，根据触发条件跳转到对应的状态，在不同的状态下做相应的处理。有限状态机主要用到 always 及 case 语句。下面以一个四状态的有限状态机举例说明。



在程序中设计了 8 位的移位寄存器，在 Idle 状态下，判断 shift_start 信号是否为高，如果为高，进入 Start 状态，在 Start 状态延迟 100 个周期，进入 Run 状态，进行移位处理，如果 shift_stop 信号有效了，进入 Stop 状态，在 Stop 状态，清零 q 的值，再跳转到 Idle 状态。

Mealy 有限状态机，输出不仅与当前状态有关，也与输入信号有关，在 RTL 中会与输入信号有连接。

```

module top
(
    input shift_start,
    input shift_stop,
    input rst,
    input clk,
    input d,
    output reg [7:0] q
);

parameter Idle = 2'd0 ; //Idle state
parameter Start = 2'd1 ; //Start state
parameter Run = 2'd2 ; //Run state
parameter Stop = 2'd3 ; //Stop state

reg [1:0] state ; //statement
reg [4:0] delay_cnt ; //delay counter

always @(posedge clk or negedge rst)
begin
    if (!rst)
    begin
        state <= Idle ;
        delay_cnt <= 0 ;
        q <= 0 ;
    
```

```

    end
  else
    case(state)
      Idle  : begin
          if (shift_start)
            state <= Start ;
          end
        Start : begin
            if (delay_cnt == 5'd99)
              begin
                delay_cnt <= 0 ;
                state <= Run ;
              end
            else
              delay_cnt <= delay_cnt + 1'b1 ;
            end
          Run  : begin
              if (shift_stop)
                state <= Stop ;
              else
                q <= {q[6:0], d} ;
              end
            Stop : begin
                q <= 0 ;
                state <= Idle ;
              end
          default: state <= Idle ;
        endcase
    end
  endmodule

```

Moore 有限状态机，输出只与当前状态有关，与输入信号无关，输入信号只影响状态的变化，不影响输出，比如对 delay_cnt 和 q 的处理，只与 state 状态有关。

```

module top
(
  input shift_start,
  input shift_stop,
  input rst,
  input clk,
  input d,
  output reg [7:0] q
);

parameter Idle  = 2'd0 ; //Idle state
parameter Start = 2'd1 ; //Start state
parameter Run   = 2'd2 ; //Run state
parameter Stop  = 2'd3 ; //Stop state

reg [1:0] current_state ; //statement
reg [1:0] next_state ;
reg [4:0] delay_cnt ; //delay counter
//First part: statement transition
always @(posedge clk or negedge rst)
begin
  if (!rst)
    current_state <= Idle ;
  else
    current_state <= next_state ;
  end
end

```

```

end
//Second part: combination logic, judge statement transition condition
always @(*)
begin
    case(current_state)
        Idle : begin
            if (shift_start)
                next_state <= Start ;
            else
                next_state <= Idle ;
        end
        Start : begin
            if (delay_cnt == 5'd99)
                next_state <= Run ;
            else
                next_state <= Start ;
        end
        Run : begin
            if (shift_stop)
                next_state <= Stop ;
            else
                next_state <= Run ;
        end
        Stop : next_state <= Idle ;
        default: next_state <= Idle ;
    endcase
end
//Last part: output data
always @(posedge clk or negedge rst)
begin
    if (!rst)
        delay_cnt <= 0 ;
    else if (current_state == Start)
        delay_cnt <= delay_cnt + 1'b1 ;
    else
        delay_cnt <= 0 ;
end

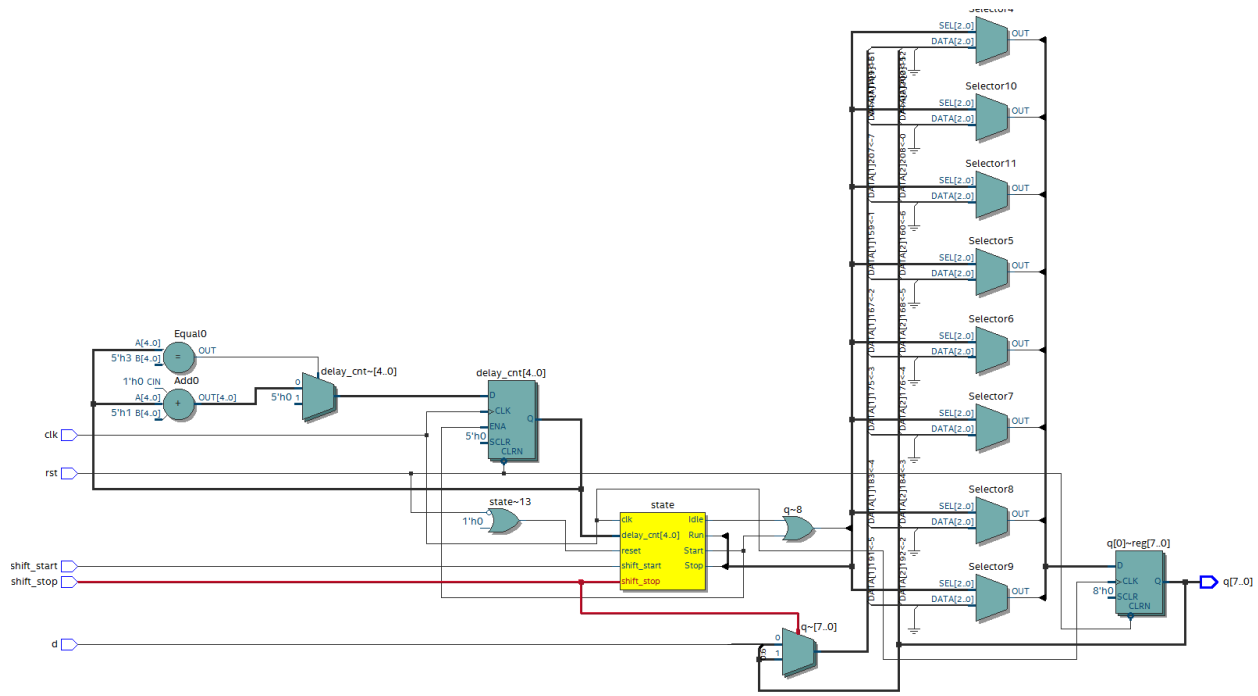
always @(posedge clk or negedge rst)
begin
    if (!rst)
        q <= 0 ;
    else if (current_state == Run)
        q <= {q[6:0], d} ;
    else
        q <= 0 ;
end

endmodule

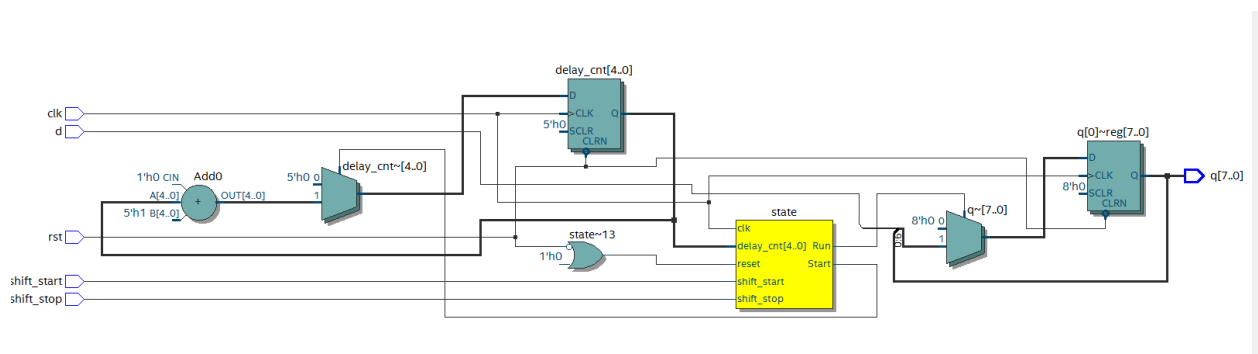
```

在上面两个程序中用到了两种方式的写法，第一种 Mealy 状态机，采用了一段式的写法，只用了一个 always 语句，所有的状态转移，判断状态转移条件，数据输出都在一个 always 语句里，缺点是如果状态太多，会使整段程序显的冗长。第二个 Moore 状态机，采用了三段式的写法，状

态转移用了一个 always 语句，判断状态转移条件是组合逻辑，采用了一个 always 语句，数据输出也是单独的 always 语句，这样写起来比较直观清晰，状态很多时也不会显得繁琐。



Mealy 有限状态机 RTL 图



Moore 有限状态机 RTL 图

激励文件如下：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg shift_start ;
reg shift_stop ;
reg rst ;
reg clk ;
reg d ;
```



```

wire [7:0] q ;

initial
begin
    rst = 0 ;
    clk = 0 ;
    d = 0 ;
    #200 rst = 1 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

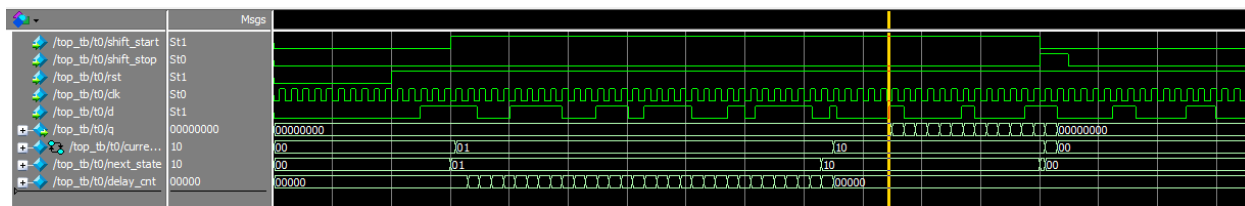
initial
begin
    shift_start = 0 ;
    shift_stop = 0 ;
    #300 shift_start = 1 ;
    #1000 shift_start = 0 ;
        shift_stop = 1 ;
    #50 shift_stop = 0 ;
end

always #10 clk = ~clk ;

top t0
(
    .shift_start(shift_start),
    .shift_stop(shift_stop),
    .rst(rst),
    .clk(clk),
    .d(d),
    .q(q)
);
endmodule

```

仿真结果如下：



6 总结

本文档介绍了组合逻辑以及时序逻辑中常用的模块，其中有限状态机较为复杂，但经常用到，希望大家能够深入理解，在代码中多运用，多思考，有利于快速提升水平。